STRIPE PROJECT

2. OLAP Data System

Annexes

2.A. Kimball Analysis

> *Nicolas Pichon --- AIA RNCP 38777 / BC02 / D3 : OLAP Annexe A / v18 --- 2026/11/05.*

2.A.1. Méthode et périmètre

La méthode **Kimball** est une méthode de conception de système OLAP (cf. *The Data Warehouse Toolkit* par *Kimball*) : le point de départ n'est pas le schéma source, mais la problématique du métier. Le processus se déroule en quatre étapes, dans l'ordre suivant :

1. **Choisir le processus métier** : un événement qui se produit dans l'entreprise.
2. **Définir la granularité des faits** : ce que représente une ligne de la table de faits.
3. **Identifier les dimensions** : les axes d'analyse.
4. **Identifier les faits** : les mesures numériques agrégeables.

Le présent document couvre le processus complet pour l'ensemble des besoins analytiques exprimés dans le cahier des charges (BR2, DS2, D3, D8), en traçant chaque choix jusqu'à sa source dans le modèle OLTP (annexe 1.B).

2.A.2. Étape 1 : Les processus métiers

Six processus métiers sont identifiés, d'après les catégories de données analytiques du cahier des charges [D0] (DS2 : *Revenue Metrics*, *Customer Segmentation Data*, *Product Performance Metrics*, *Fraud Analysis Data*, *Compliance and Audit Logs*).

#	Processus métier	Événement déclencheur	Catégorie couverte
P1	Traitement d'un paiement	Une transaction est créée	Revenue Metrics, Product Performance, Customer Segmentation
P2	Évaluation du risque de fraude	Un score de fraude est calculé	Fraud Analysis Data
P3	Cycle de vie d'un abonnement	Un abonnement atteint une échéance	Revenue Metrics (récurrent)
P4	Accès et modification de données	Une consultation ou une modification est journalisée	Compliance and Audit Logs
P5	Traitement d'une demande de droit	Un client exerce un droit RGPD/CCPA	Compliance and Audit Logs
P6	Traitement d'un incident de sécurité	Un incident est détecté	Compliance and Audit Logs

Problématiques volontairement exclues : les remboursements (Refund) et rétro-facturations (ChargeBack) ne constituent pas des processus séparés. Ce sont des *attributs de l'issue* d'une transaction (P1), et non des événements analytiques autonomes (cf. BR2).

2.A.3. Étape 2 : Les grains

Le grain fixe ce qu'une ligne de fait représente, et ne doit jamais être mélangé avec un autre grain dans la même table (risque de double comptage lors des agrégations).

Fait	Grain	Type de table (Kimball)
`FactTransaction`	Une transaction individuelle	Fait transactionnel
`FactFraudEvent`	Une évaluation de risque de fraude	Fait transactionnel
`FactSubscriptionSnapshot`	Un abonnement, à la fin de chaque période de facturation	Instantané périodique
`FactAuditEvent`	Un événement d'audit (accès **ou** modification)	Fait transactionnel
`FactDataSubjectRequest`	Une demande de droit, mise à jour au fil du traitement	Instantané cumulatif
`FactSecurityIncident`	Un incident de sécurité, mis à jour au fil du traitement	Instantané cumulatif

Faits transactionnels et mises à jour. `FactTransaction` et `FactFraudEvent` sont insérés une fois, au grain ci-dessus. Seules quelques colonnes d'indicateurs sont mises à jour ensuite, parce que l'information arrive après l'événement (remboursement, litige, décision de fraude confirmée) : la liste exhaustive est donnée par la règle ETL-02 (2.A.9.1). Les clés, `date_key` et les montants de la transaction ne changent pas.

Pourquoi `DataAccessLog` et `ChangeAuditLog` fusionnent-ils dans `FactAuditEvent` ?
Tables distinctes dans le système OLTP, elle correspond au même type d'évènement dans le système OLAP car ils donnent la même réponse à la question : *"qui a fait quoi, quand, sur quoi"*. Un grain unique sert mieux le reporting consolidé de conformité, sans que cela contredise la séparation OLTP.

Pourquoi `FactDataSubjectRequest` et `FactSecurityIncident` sont-ils des instantanés cumulatifs et non des faits transactionnels classiques. Une demande de droit ou un incident traverse plusieurs jalons dans le temps (`received_at` --> `due_at` --> `fulfilled_at` ; `detected_at` --> `notified_at` --> `resolved_at`).
Le fait cumulatif Kimball est justement conçu pour ce cas : une ligne par demande/incident, mise à jour à chaque jalon franchi, avec une clé de date par jalon (qui)permet de calculer directement des délais de traitement sans reconstruire l'historique à partir d'événements séparés).

Pourquoi `FactSubscriptionSnapshot` est-il un instantané périodique.
Un abonnement ne génère pas d'événement ponctuel quotidien. C'est son état (actif, montant, échéance) que l'on veut pouvoir suivre dans le temps. C'est l'usage typique de l'instantané périodique : une ligne par abonnement et par point de mesure (ici, la fin de chaque cycle de facturation, cohérent avec `Subscription.current_period_end`).

2.A.4. Étape 3 - Les dimensions

Les dimensions sont *conformées* : une même dimension (ex. `DimMerchant`) est partagée par plusieurs faits, avec les mêmes clés et attributs, pour permettre des analyses transverses (ex. comparer le revenu et les incidents de sécurité par marchand).
C'est la matrice de bus ci-dessous qui documente ce partage.

Dimension	Type SCD	Origine OLTP	Remarque
`DimDate`	- (statique)	Calendrier généré	Partagée par tous les faits
`DimMerchant`	SCD2	`Merchant` + `MerchantStatus` + `Country`	Historise les changements de statut/pays
`DimCustomer`	SCD2	`Customer` + `Country`	Historise le pays de résidence
`DimProduct`	SCD1	`Product`	Pas d'historisation : le prix courant suffit, le montant réel est déjà figé dans le fait
`DimPaymentMethod`	SCD1	`PaymentMethod` + `PaymentMethodType` + `CardBrand`	Type et marque, jamais le PAN
`DimPricingPlan`	SCD2	`PricingPlan`	Doit s'historiser : un plan corrigé rétroactivement ne doit pas réécrire le passé (cf. `Transaction.pricing_plan_id` et les règles MCD #22, #23, #24, #25 de l'OLTP) ; les frais fixes par devise (`PricingPlanFee`) ne sont pas repris
`DimCurrency`	- (statique)	`Currency`	
`DimGeography`	SCD1 (statique)	`Country` (référentiel) + `Transaction.ip_geolocation` (pays résolu à l'ETL) + `Transaction.device_type`	Pays de la **transaction**, pas celui du client ni du marchand. Dénormalisation assumée (deux notions combinées) ; membre « unknown » ; voir ci-dessous
`DimTransactionStatus`	- (statique)	`TransactionStatus`	Inclut `allows_refund` , `is_terminal`
`DimFraudRiskLevel`	- (statique)	`FraudRiskLevel`	Inclut `requires_manual_review` , `blocks_transaction`
`DimSubscriptionStatus`	- (statique)	`SubscriptionStatus`	Inclut `triggers_dunning`
`DimActor`	SCD2	`AuditActorType` (OLTP) + `actor_id` des journaux `DataAccessLog` / `ChangeAuditLog`	Historise le type d'acteur si réaffecté
`DimAuditAction`	- (statique)	`AuditAction`	Inclut `is_sensitive` , `requires_justification`
`DimRegulation`	- (statique)	`Regulation`	Inclut `response_deadline_days`
`DimRequestStatus`	- (statique)	`DataSubjectRequestStatus`	
`DimIncidentStatus`	- (statique)	`SecurityIncidentStatus`	

Dimension	Type SCD	Origine OLTP	Remarque
DimReferenceExchangeRate	Versionnée (annuelle)	Constante de référence, hors OLTP	Voir 2.A.7

Dimensions indépendantes. Aucune dimension ne référence une autre dimension : seules les tables de faits portent des clés étrangères vers les dimensions (étoile pure, D3.2). DimMerchant étant en SCD2, une clé étrangère DimCustomer → DimMerchant désignerait une version du marchand et obligerait à versionner tous ses clients, produits et plans tarifaires à chaque changement du marchand. DimCustomer , DimProduct et DimPricingPlan portent donc merchant_id (clé métier de l'OLTP, sans lien) pour le filtrage RLS, et DimReferenceExchangeRate identifie la devise source par from_currency_code . Le marchand d'une ligne de fait reste merchant_key du fait ; l'ETL vérifie que le merchant_id du client, du produit ou du plan est celui de ce merchant_key .

Pourquoi DimProduct reste-t-elle en SCD1 alors que DimMerchant et DimCustomer sont en SCD2. Le montant réellement payé est déjà figé dans FactTransaction.amount au moment de la vente. Retrouver le prix catalogue *historique* d'un produit n'apporte rien que le fait ne porte déjà. DimMerchant / DimCustomer , en revanche, sont interrogées pour des attributs qui *évoluent indépendamment* du fait (statut, pays) et que l'on veut pouvoir dater précisément.

Lignée de DimGeography . La dimension décrit *où* et *avec quel appareil* la transaction a été initiée, et non *qui* est le client :

- **Pays** : Transaction.ip_geolocation contient l'adresse IP (IPv4/IPv6) de l'appelant. Le nom de la colonne est conservé dans l'OLTP, bien qu'il désigne une adresse IP et non un pays (la note de la colonne le précise). L'ETL la résout en pays avec une base GeoIP (version figée au chargement, pour qu'un rechargement donne le même résultat), puis rattache ce pays à Country pour obtenir le nom et la région.
- **Appareil** : Transaction.device_type (mobile , desktop , tablet).
- **Non résolu** : IP absente, privée, non résolue ou pays hors Country → membre par défaut « unknown » (country_code = 'ZZ' , country_name = 'unknown' , region = 'unknown' , device_type = 'unknown'). Les clés geography_key de FactTransaction et FactFraudEvent sont donc non nulles (une clé nulle ferait disparaître ces lignes des analyses par jointure interne).
- **RGPD** : l'adresse IP est une donnée personnelle ; elle n'est pas copiée dans l'OLAP, seul le pays l'est.
- **Pays du client et du marchand** : ils restent dans DimCustomer.country_code et DimMerchant.country_code (SCD2). Comparer ces pays au pays de la transaction permet d'identifier les transactions transfrontalières ou à pays discordant, utile en analyse de fraude.
- **Type SCD** : statique (SCD1). Une ligne par couple (pays, appareil) ; les changements de référentiel Country (nom, région) écrasent la valeur, sans historique.
- FactAuditEvent : geography_key est non nulle. Pour les accès (DataAccessLog), le pays est résolu depuis source_ip , l'adresse d'origine de la requête, comme pour Transaction.ip_geolocation . Pour les modifications (ChangeAuditLog , alimenté par CDC), l'OLTP v36 ne porte ni IP ni pays : ces lignes reçoivent le membre « unknown ». device_type vaut toujours unknown pour ce fait. Une analyse par pays ne couvre donc que les accès. Pour couvrir aussi les modifications, il faudrait porter l'adresse source dans ChangeAuditLog côté OLTP ; ce n'est pas retenu en l'absence de besoin.

2.A.5. Étape 4 : Les faits

Table de faits	Mesures	Nature
FactTransaction	Montants locaux en unités mineures : amount_minor , fee_amount_minor , refund_amount_minor ; montants en EUR : amount_eur , fee_amount_eur , stripe_revenue_eur (dérivée de fee_amount_eur), refund_amount_eur , merchant_net_revenue_eur (calculé) ; indicateurs : is_refunded , is_chargeback , is_merchant_initiated ; score de fraude aplati : anomaly_score	Additives sur toutes les dimensions sauf is_* (semi-additives) ; anomaly_score non additif (moyenne pertinente, somme non) ; les montants locaux ne s'additionnent qu'à devise constante
FactFraudEvent	anomaly_score , is_confirmed_fraud , exposed_amount_eur ; le niveau de risque, la revue manuelle et le blocage sont portés par DimFraudRiskLevel (requires_manual_review , blocks_transaction)	anomaly_score non additif (moyenne pertinente, somme non) ; exposed_amount_eur additif ; is_confirmed_fraud semi-additif
FactSubscriptionSnapshot	mrr_eur (revenu récurrent mensualisé, calculé), stripe_mrr_eur (part de commission revenant à Stripe), failed_attempt_count , is_active , days_until_renewal	mrr_eur et stripe_mrr_eur semi-additifs (additif sur toutes les dimensions sauf le temps - sommer deux instantanés du même abonnement à deux dates n'a pas de sens)
FactAuditEvent	was_authorized , had_justification ; comptage par COUNT(*) (une ligne = un événement, pas de colonne event_count) ; la sensibilité et l'obligation de justification sont portées par DimAuditAction (is_sensitive , requires_justification)	Comptage entièrement additif

Table de faits	Mesures	Nature
`FactDataSubjectRequest`	`processing_days` (calculé, `fulfilled_at − received_at`), `is_within_deadline` (le retard éventuel se déduit de `processing_days` et de `DimRegulation.response_deadline_days`, il n'est pas stocké)	Non additif (moyenne/max pertinents, jamais de somme)
`FactSecurityIncident`	`affected_record_count`, `detection_to_notification_hours`, `is_within_72h_deadline`	`affected_record_count` additif, les délais non additifs

Les règles de calcul de ces mesures et indicateurs sont réunies en 2.A.9.

2.A.6. Matrice de bus (Kimball Bus Matrix)

Vue synthétique du partage des dimensions entre processus.
Chaque case cochée est une dimension conformée, réutilisable telle quelle d'un fait à l'autre.

Dimension / Processus	P1 Transaction	P2 Fraude	P3 Abonnement	P4 Audit	P5 Droit RGPD	P6 Incident
DimDate	×	×	×	×	×	×
DimMerchant	×	×	×	×	×	×
DimCustomer	×	×	×	×	×	
DimProduct	×		×			
DimPaymentMethod	×					
DimPricingPlan	×					
DimCurrency	×					
DimGeography	×	×		×		
DimTransactionStatus	×					
DimFraudRiskLevel		×				
DimSubscriptionStatus			×			
DimActor				×		
DimAuditAction				×		
DimRegulation					×	
DimRequestStatus					×	
DimIncidentStatus						×

`DimDate` et `DimMerchant` traversent les six processus. C'est ce qui permettra, par exemple, de croiser revenu et incidents de sécurité par marchand et par mois sans jointure artificielle entre deux entrepôts séparés.

2.A.7. Conversion de devise

`DimReferenceExchangeRate` porte un taux de conversion de référence fixe pour l'année (différent a priori du taux du jour) pour que deux montants convertis à des dates différentes restent comparables entre eux. La devise de référence (EUR) est une constante du modèle : aucune colonne ne la porte (ni `to_currency_*` dans `DimReferenceExchangeRate`, ni drapeau dans `DimCurrency`).

2.A.7.1. Représentation des montants

- **Montants locaux** : entiers en unités mineures (`bigint`), comme dans l'OLTP (`Transaction.amount`, `Transaction.fee_amount`, `Refund.amount`). L'échelle est portée par `DimCurrency.decimals` (JPY = 0, EUR = 2, BHD = 3) : montant en unité majeure = `amount_minor / 10^decimals`. Aucune perte, quelle que soit la devise, et rapprochement exact avec l'OLTP.
- **Montants en EUR** : `decimal(18,4)`, calculés une seule fois à l'ETL, en décimal exact (jamais en flottant).

2.A.7.2. Formule de conversion

`amount_eur = amount_minor / 10^decimals × reference_rate`

où `reference_rate` (`decimal(24,12)`) est exprimé en **EUR pour 1 unité majeure** de la devise source. Exemples (taux fictifs, à titre d'illustration) :

Devise	`amount_minor`	`decimals`	Montant local	`reference_rate`	`amount_eur`
BHD	12345	3	12,345	2,4	29,6280

Devise	amount_minor	decimals	Montant local	reference_rate	amount_eur
JPY	5000	0	5 000	0,0062	31,0000
EUR	1990	2	19,90	1	19,9000

2.A.7.3. Règles d'application

1. **Taux retenu** : celui de (devise source, exercice) pour la date de création UTC de la transaction. La paire EUR → EUR (taux 1) existe, afin que `FactTransaction.exchange_rate_key` reste non nul.
2. **Une seule conversion par composante** : montant, commission et remboursement sont convertis chacun une fois. La commission (`fee_amount_minor`) est reprise telle quelle de `Transaction.fee_amount` ; elle n'est **jamais recalculée** côté OLAP. `stripe_revenue_eur` est une colonne dérivée de `fee_amount_eur` (`GENERATED ALWAYS AS (fee_amount_eur) STORED`), jamais alimentée séparément. Les mesures dérivées sont calculées sur les valeurs déjà converties : `merchant_net_revenue_eur = amount_eur − fee_amount_eur − refund_amount_eur` (identité exacte).
3. **Arrondi** : à la 4e décimale, demi-supérieur. L'arrondi à 2 décimales est réservé à l'affichage, pour que les agrégats sur de grands volumes ne cumulent pas de biais d'arrondi.
4. **Rapprochement** : `SUM(amount_minor)` par jour, marchand et devise doit égaler la somme des `Transaction.amount` correspondantes de l'OLTP.

2.A.8. Fuseaux horaires

Règle : tout est en UTC. L'OLTP stocke ses horodatages en `timestamptz` (UTC) ; l'OLAP reprend cette référence, sans conversion en heure locale.

Élément OLAP	Type	Règle
`FactTransaction.transaction_timestamp` , `FactAuditEvent.event_timestamp`	`timestamptz`	Valeur UTC. Un `timestamp` sans fuseau est interdit : il ne dit pas quelle heure il représente.
`date_key` (et `*_date_key`) de chaque fait	`int` AAAAMMJJ	Date **UTC** de l'événement (tableau ci-dessous).
`valid_from` , `valid_to` (dimensions SCD2), `effective_from` , `effective_to` (taux)	`date`	Dates UTC.
Exercice fiscal (`DimReferenceExchangeRate.fiscal_year`)	`smallint`	Bornes en UTC ; c'est la même date que celle qui sélectionne le taux (2.A.7.3, règle 1).

Origine de la date de chaque fait :

Fait	`date_key` calculée à partir de
`FactTransaction`	`Transaction.created_at`
`FactFraudEvent`	`FraudScore.evaluated_at`
`FactSubscriptionSnapshot`	`date_key` : date de l'instantané (fin de la période de facturation en cours) ; `started_date_key` : `Subscription.started_at` ; `canceled_date_key` : `Subscription.canceled_at` (demande de résiliation, NULL si aucune)
`FactAuditEvent`	`DataAccessLog.occurred_at` ou `ChangeAuditLog.occurred_at`
`FactDataSubjectRequest`	`DataSubjectRequest.received_at` (réception), `fulfilled_at` (traitement)
`FactSecurityIncident`	`SecurityIncident.detected_at` , `notified_at` , `resolved_at`

Pourquoi UTC plutôt que l'heure du marchand ?

- Une seule définition du « jour » pour tous les marchands : les agrégats (`AggRevenueDailyMerchant` , etc.) restent additionnables et rapprochables de l'OLTP.
- Le taux de change et l'exercice fiscal se déterminent avec la même date que `date_key` : pas de transaction dont le jour diffère selon la table lue.
- Le modèle OLTP ne porte aucun fuseau par marchand ; sans fuseau IANA fiable, un pays à plusieurs fuseaux interdit de déduire l'heure locale du pays seul.

Conséquences à connaître. Un marchand loin de l'Europe verra une transaction passée à 23 h 30 heure locale comptée au jour UTC suivant ou précédent. Pour un besoin de clôture quotidienne en heure locale, il faudrait ajouter `Merchant.timezone` (fuseau IANA) à l'OLTP et un `local_date_key` aux faits ; cette extension n'est pas retenue en l'absence de besoin exprimé.

Requêtes. « Aujourd'hui » s'écrit `(now() AT TIME ZONE 'UTC')::date` ; `CURRENT_DATE` dépend du fuseau de la session et est donc proscrit (corrigé en P1.1, P4.1 et P5.2 de D8.2). Pour une analyse par heure, utiliser `EXTRACT(hour FROM transaction_timestamp AT TIME ZONE 'UTC')` ; aucune dimension `DimTime` n'est créée.

Jours fériés. `DimDate` ne porte aucun pays, et OLTP ne fournit aucune source de jours fériés.

2.A.9. Règles de transformation (ETL)

Cette section réunit les règles qui produisent les valeurs que l'OLTP ne fournit pas telles quelles : indicateurs, mesures calculées, regroupements, segments. Chaque règle porte un identifiant (`ETL-nn`) cité dans les notes du modèle physique [D3-B]. Les points non tranchés sont exposés en 2.A.9.8 (`OUV-nn`). Les règles sur les montants (2.A.7) et sur les fuseaux horaires (2.A.8) restent à leur place et s'appliquent à toutes celles ci-dessous.

Statut des valeurs. Les sources (tables et colonnes de l'OLTP v36) sont celles du MCD et du modèle physique OLTP. Les seuils, les mappings et les choix signalés « *hypothèse de conception* » sont des décisions de ce document, non des faits issus de l'OLTP ; ils sont à valider avec le besoin métier.

2.A.9.1. Principes communs

ID	Règle
ETL-01	**Version de dimension SCD2 retenue.** Pour un fait daté du jour UTC `D` , l'ETL retient la version de la dimension telle que `valid_from ≤ D < valid_to` (`valid_to` NULL = version courante). Les bornes sont demi-ouvertes, comme les `daterange` de l'OLTP (`PricingPlan`) : `valid_to` d'une version est égal à `valid_from` de la suivante.
ETL-02	**Colonnes mises à jour après l'insertion.** Les faits sont insérés une fois. Seules exceptions, car l'information arrive après la transaction : `FactTransaction.status_key` , `refund_amount_minor` , `refund_amount_eur` , `merchant_net_revenue_eur` , `is_refunded` , `is_chargeback` ; `FactFraudEvent.is_confirmed_fraud` . Les clés, `date_key` , les montants de la transaction et la commission ne changent jamais.
ETL-03	**Rechargement idempotent.** Un fait rechargé donne la même ligne : clé métier unique (`transaction_id` , `subscription_id` + date d'instantané), taux de référence (2.A.7) et base GeoIP (2.A.4) figés.

2.A.9.2. Dimensions

ID	Cible	Règle
ETL-10	`DimCustomer.segment`	Évalué **le 1er jour de chaque mois (UTC)**, pour chaque client, dans cet ordre : (1) `nouveau` si `Customer.created_at` date de moins de 90 jours ; (2) `dormant` si, sinon, aucune transaction aboutie (statuts `successful` ou `refunded`) sur les 180 derniers jours ; (3) `VIP` si, sinon, la somme de `amount_eur` des transactions abouties des 12 derniers mois place le client parmi les 5 % supérieurs de son marchand (seulement si le marchand compte au moins 20 clients actifs, sinon on passe à (4)) ; (4) `régulier` dans tous les autres cas. Une nouvelle version SCD2 (`valid_from` = jour d'évaluation) n'est créée que si le segment change. *Hypothèse de conception : 90 jours, 180 jours, 5 %, 20 clients.* Aucune de ces valeurs ne vient de l'OLTP.
ETL-11	`DimAuditAction.action_group`	Correspondance fixe sur `AuditAction.action_code` : `read` , `export` , `login` → `lecture` (actions du journal des accès, `DataAccessLog`) ; `create` , `update` , `delete` → `ecriture` ; `grant` , `revoke` → `administration` (modification d'un état d'autorisation). `login` est classé `lecture` parce qu'il est journalisé dans `DataAccessLog` ; il n'est pas une lecture de données : une analyse « lectures de données » l'exclut par `action_code` . Un `action_code` inconnu interrompt le chargement de la dimension (pas de groupe par défaut).

Pourquoi un recalcul mensuel pour le segment. Un recalcul quotidien créerait une version SCD2 chaque fois qu'un client franchit un seuil, et le fait se rattache à la version en vigueur le jour de la transaction (ETL-01) : le segment d'une transaction est donc celui du début de mois. Une mini-dimension de segments est écartée : elle serait une seconde dimension liée au fait pour un seul attribut, alors que les dimensions restent indépendantes (2.A.4).

2.A.9.3. `FactTransaction`

ID	Colonne	Règle
ETL-20	`is_refunded` , `refund_amount_minor` , `refund_amount_eur`	`is_refunded` est vrai si la transaction a un `Refund` (au plus un, règle MCD #13), y compris pour un remboursement partiel. `refund_amount_minor = Refund.amount` (0 sinon) ; `refund_amount_eur` est converti une seule fois, au taux de la transaction (2.A.7.3). Le statut `refunded` de `Transaction` n'est pas la source ; un désaccord entre ce statut et l'existence d'un `Refund` est signalé comme anomalie de chargement, non corrigé.

ID	Colonne	Règle
ETL-21	`is_chargeback`	Vrai si la transaction a un `Chargeback` (au plus un, règle MCD #14), **quel que soit son statut** (`open` , `won` , `lost`) : un litige gagné par le marchand reste un litige. Le fait ne porte ni le statut ni un montant perdu ; `merchant_net_revenue_eur` n'est donc pas diminué par une rétrofacturation.
ETL-22	`anomaly_score`	`FraudScore.anomaly_score` de la transaction ; NULL si elle n'a pas été évaluée.

La date d'un remboursement ou d'un litige n'est pas portée par le fait : `date_key` reste la date de la transaction.

2.A.9.4. `FactFraudEvent`

ID	Colonne	Règle
ETL-30	ligne, `risk_level_key` , `anomaly_score` , `date_key`	Une ligne par `FraudScore` (1:1 avec la transaction). `risk_level_key` désigne `FraudScore.risk_level_code` ; `anomaly_score` est copié ; `date_key` est la date UTC de `FraudScore.evaluated_at` .
ETL-31	`exposed_amount_eur`	Reprise de `FactTransaction.amount_eur` de la transaction évaluée : même valeur, même taux, aucune nouvelle conversion, donc les deux faits concordent.
ETL-32	`is_confirmed_fraud`	Faux à l'insertion. Devient vrai quand la transaction a une `FraudDecision` avec `decision_value = 'fraud'` **et** `decision_status = 'confirmed'` (règle MCD #41). Une décision `defaulted` (décision automatique faute de revue), même « fraud », n'est pas une fraude confirmée ; une décision `legitimate` non plus. La colonne suit l'état courant de la décision. Une rétrofacturation, même de catégorie `fraud` , n'est pas une source : elle ne vaut pas décision.

2.A.9.5. `FactSubscriptionSnapshot`

Les valeurs sont celles de l'abonnement à l'instant de l'instantané, lues après la tentative de prélèvement de fin de période.

ID	Colonne	Règle
ETL-40	`mrr_eur`	0 si le statut n'est pas facturable (`DimSubscriptionStatus.is_billable = false`). Sinon, `ROUND( Subscription.amount / 10^decimals × reference_rate × f , 4 )` , avec `f` = 52/12 (`weekly`), 1 (`monthly`), 1/12 (`yearly`) d'après `Subscription.billing_cycle` . Le taux est celui de (devise de l'abonnement, exercice de la date de l'instantané). Un seul arrondi, à la fin, demi-supérieur (2.A.7.3).
ETL-41	`stripe_mrr_eur`	Même calcul que `mrr_eur` , appliqué à `Transaction.fee_amount` de la **dernière transaction aboutie** de l'abonnement (`Transaction.subscription_id`) à la date de l'instantané, dans la devise de cette transaction. La commission est reprise, non recalculée. 0 si le statut n'est pas facturable ou s'il n'y a pas encore de transaction aboutie. *Limite :* après un changement de plan tarifaire, la valeur ne bouge qu'au prélèvement suivant.
ETL-42	`is_active`	Vrai si `Subscription.status_code = 'active'` . `paused` , `past_due` et `canceled` donnent faux. Une demande de résiliation ne change rien avant la fin de période (règle MCD #20).
ETL-43	`days_until_renewal`	`Subscription.current_period_end` (jour UTC) moins `date_key` , si cette date est postérieure à `date_key` ; NULL sinon (abonnement non renouvelé, ou `current_period_end` absente).
ETL-44	`started_date_key` , `canceled_date_key`	Jour UTC de `Subscription.started_at` ; jour UTC de `Subscription.canceled_at` , NULL tant qu'aucune demande n'existe. Valeur connue à la date de l'instantané : les instantanés antérieurs à la demande gardent NULL.

2.A.9.6. `AggSubscriptionDailyMerchant`

Grain : jour `D` (UTC) × marchand × produit. Les compteurs comptent des abonnements distincts (`subscription_id`).

ID	Colonne	Règle
ETL-50	`new_subscriptions`	Abonnements dont `started_date_key = D` .
ETL-51	`churned_count`	Abonnements dont `canceled_date_key = D` : ce sont des **demandes de résiliation**. Les fins d'abonnement par échecs de prélèvement répétés (`canceled_at` vide, règle MCD #20) ne sont pas comptées.
ETL-52	`active_subscriptions` , `past_due_subscriptions`	État au jour `D` : le dernier instantané de l'abonnement dont `date_key ≤ D` est reporté jusqu'à l'instantané suivant. `active_subscriptions` : `is_active` vrai ; `past_due_subscriptions` : `DimSubscriptionStatus.triggers_dunning` vrai.

ID	Colonne	Règle
ETL-53	`churn_rate`	`churned_count(D) / active_subscriptions(D − 1)` , arrondi à la 4e décimale (demi-supérieur) ; NULL si le dénominateur est 0.

Compteurs provisoires. Un abonnement n'apparaît dans le fait qu'à son premier instantané, à la fin de sa première période : pour un abonnement annuel, `new_subscriptions` du jour de souscription ne devient exact qu'un an plus tard. Il en va de même de `churned_count` pour une demande de résiliation faite entre deux instantanés. L'ETL recalcule les jours concernés à chaque chargement d'instantanés ; les compteurs des cycles récents sont donc à lire comme provisoires. Pour des compteurs exacts le jour même, il faudrait produire aussi un instantané à la souscription et à la demande de résiliation, ce qui change le grain actuel (2.A.3) ; ce n'est pas retenu à ce stade.

2.A.9.7. `FactAuditEvent`

ID	Source	Règle
ETL-60	`DataAccessLog`	Une ligne par entrée. `audit_action_key` d'après `action_code` ; `actor_key` : version de (`actor_id` , `actor_type_code`) en vigueur à la date de l'événement (ETL-01) ; `merchant_key` et `customer_key` : NULL si l'entrée n'en désigne pas (règle MCD #29) ; `geography_key` : pays résolu depuis `source_ip` (2.A.4) ; `resource_type` copié ; `was_authorized` copié ; `had_justification` vrai si `justification` n'est ni NULL ni vide.
ETL-61	`ChangeAuditLog`	Une ligne par entrée. `merchant_key` : celui de l'entrée, NULL s'il n'y en a pas ; `customer_key` : **toujours NULL** (une écriture ne vise jamais un client isolément, règle MCD #31) ; `geography_key` : membre « unknown » ; `resource_type = table_name` ; `was_authorized` = vrai (le journal ne contient que des écritures exécutées, pas de refus) ; `had_justification` = faux, ce qui signifie ici « non journalisée » et non « injustifiée » : l'OLTP ne conserve pas de motif sur ce journal. Les analyses de justification se limitent donc aux lignes `action_group = 'lecture'` (ETL-11).
ETL-62	contrôle	Quand une entrée de `DataAccessLog` désigne un client et un marchand, le client doit appartenir à ce marchand (règle MCD #33). L'ETL signale l'écart comme anomalie de chargement ; il ne le corrige pas.

2.A.9.8. Politiques et règles de transformation à définir

Les points ci-dessous ne sont **pas tranchés**. Pour chacun, le tableau expose la situation actuelle du modèle et ce qui reste à définir ; il ne propose pas de solution. Les valeurs retenues provisoirement ailleurs dans 2.A.9 y sont signalées.

ID	Sujet	Situation actuelle	À définir
OUV-01	Date d'un remboursement ou d'un litige	`FactTransaction` ne porte que la date de la transaction (`date_key`). `is_refunded` , `refund_amount_eur` et `is_chargeback` sont donc rattachés au jour de la transaction (ETL-20, ETL-21), et `AggRevenueDailyMerchant.refund_total_eur` en reprend la date.	Si le reporting doit situer un remboursement ou un litige à sa propre date, et, dans ce cas, comment la date est portée par le modèle.
OUV-02	Commission en cas de remboursement	`fee_amount_minor` est repris de l'OLTP et un remboursement ne le modifie pas (ETL-02, ETL-20). `stripe_revenue_eur` reste donc égal à `fee_amount_eur` , et `merchant_net_revenue_eur = amount_eur − fee_amount_eur − refund_amount_eur` laisse la commission à la charge du marchand, y compris pour un remboursement partiel.	La politique applicable à la commission d'une transaction remboursée (conservée, restituée en tout ou en partie, selon le type de remboursement) et la manière dont le modèle la reflète.
OUV-03	Grain de l'instantané d'abonnement	Un instantané est produit à la fin de chaque période de facturation (2.A.3). Un abonnement entre donc dans le fait à la fin de sa première période, et `new_subscriptions` ainsi que `churned_count` sont provisoires jusqu'à l'instantané suivant (2.A.9.6).	Si des instantanés doivent aussi être produits à la souscription ou à la demande de résiliation, et l'effet de ce choix sur le grain décrit en 2.A.3.
OUV-04	Définition du churn	`churned_count` compte les demandes de résiliation (`canceled_date_key` , ETL-51). Les fins d'abonnement par échecs de prélèvement répétés (`canceled_at` vide, règle MCD #20) ne sont pas comptées. `churn_rate` rapporte ces demandes aux abonnements actifs de la veille (ETL-53).	Ce que « churn » désigne (demande de résiliation, fin effective, ou les deux), le traitement des fins sans demande, et le dénominateur du taux.
OUV-05	`had_justification` pour `ChangeAuditLog`	La colonne est `not null` ; pour une ligne issue de `ChangeAuditLog` elle vaut faux, ce qui signifie «	Comment représenter « information absente » distinctement de « pas de justification », et comment les analyses de justification

ID	Sujet	Situation actuelle	À définir
		non journalisée » et non « injustifiée » (ETL-61), car l'OLTP ne conserve pas de motif sur ce journal.	(`DimAuditAction.requires_justification`) en tiennent compte.
OUV-06	`DimActor.is_human`	La colonne est `not null` avec la valeur par défaut faux. Dans l'OLTP, `AuditActorType.is_human` est indéterminé pour le type `merchant_api` .	Comment représenter un acteur dont la nature humaine ou automatique est indéterminée.
OUV-07	Valeurs retenues provisoirement par 2.A.9	Aucune de ces valeurs ne vient de l'OLTP : seuils du segment client (90 jours, 180 jours, 5 %, 20 clients ; ETL-10) ; `login` classé `lecture` (ETL-11) ; `is_chargeback` vrai pour tous les statuts de litige (ETL-21) ; `is_confirmed_fraud` défini par la décision de fraude confirmée (ETL-32), en remplacement de l'ancienne mention d'une rétrofacturation ; `stripe_mrr_eur` pris sur la commission de la dernière transaction aboutie (ETL-41) ; `mrr_eur` nul pour un statut non facturable, selon des valeurs de `is_billable` qui sont des données de référence (ETL-40) ; `is_active` vrai pour le seul statut `active` (ETL-42).	La validation métier de chacune de ces valeurs.
OUV-08	Visibilité des lignes sans marchand	`FactAuditEvent.merchant_key` et `FactSecurityIncident.merchant_key` sont nullables (accès ou incident transverses à la plateforme). La sécurité au niveau des lignes porte sur `merchant_key` (D3.5.7) ; son effet sur les lignes à clé NULL n'est pas précisé.	Qui peut lire les lignes sans marchand, et la politique correspondante.
OUV-09	Règles ETL manquantes sur les faits	Aucune règle de transformation n'est écrite pour : `FactDataSubjectRequest` (`processing_days` , `is_within_deadline` , mise à jour à chaque jalon, `request_status_key`) ; `FactSecurityIncident` (`detection_to_notification_hours` , `is_within_72h_deadline` , jalons, `incident_status_key`) ; `FactTransaction.is_merchant_initiated` et la mise à jour de `status_key` après l'insertion.	Les règles de calcul et de mise à jour de ces colonnes.
OUV-10	Règles ETL manquantes sur les agrégats	Seules les colonnes de `AggSubscriptionDailyMerchant` portant sur les compteurs et le taux de churn ont des règles (2.A.9.6). Restent sans règle : `merchant_mrr_eur` et `stripe_mrr_eur` de cette table (MRR brut ou net de la commission) ; `AggRevenueDailyMerchant` ; `AggRevenueMonthlyMerchantCountry` (le pays visé par `country_code` : transaction, client ou marchand) ; `AggFraudDailyMerchant` (`high_risk_count` , `confirmed_fraud_count` , `fraud_rate`) ; `AggComplianceDailyMerchant` (`total_access_count` , `sensitive_access_count` , `unauthorized_attempt_count` , `distinct_actor_count` , `export_count`) ; `AggDataSubjectRequestMonthly` et `AggSecurityIncidentMonthly` (mois de rattachement).	Les règles de calcul, le jour ou le mois de rattachement et la fréquence de recalcul de chaque agrégat.
OUV-11	Provenance et chargement des dimensions	2.A.4 indique l'origine OLTP de chaque dimension, mais aucune règle n'en décrit le chargement : création des versions SCD2 de `DimMerchant` , `DimCustomer` (hors segment), `DimPricingPlan` et `DimActor` (l'identité des acteurs est gérée hors du modèle, règle MCD #28) ; génération de `DimDate` (plage de dates) ; chargement annuel de `DimReferenceExchangeRate` (source, reprise de l'historique) ; mise à jour des dimensions statiques.	Les règles de chargement de chaque dimension.

Références

Spécifications

- [D0] Stripe Project Introduction